

Java vs JavaScript — 코딩테스트 관점 비교

같은 문제를 두 언어로 풀 때 혼동하기 쉬운 차이점과 대응 관계를 정리한다.

1. 대응표

Java	JavaScript	역할
ArrayList	Array	순서 있는 동적 배열
HashMap	Object / Map	Key-Value 저장소
Comparator	sort() 콜백	정렬 기준 정의
Stream API	고차 함수 (filter/map/reduce)	배열 처리 파이프라인
HashSet	Set	중복 제거
Iterator	for...of / Symbol.iterator	순회 메커니즘
Optional	?. / ?? / undefined 체크	null 안전 처리

2. 핵심 차이점

2-1. 언어 설계 철학의 차이

Java와 JavaScript는 설계 철학이 다르고, 이 차이가 자료구조, 문법, 코딩테스트 패턴 전반에 영향을 준다.

	Java	JavaScript
설계 철학	성능과 타입 안정성 우선	편의성 우선
배열 구조	int[] (고정) + ArrayList (동적) + Stream (처리) 분리	Array 하나가 모두 담당
내부 동작	개발자가 직접 성능과 편의를 선택	엔진이 상황에 따라 알아서 최적화
성능 보장	고정 배열은 연속 메모리로 빠른 접근 보장	타입이 같으면 연속 메모리, 섞이면 해시맵 구조로 전환
타입	선언 필수 (컴파일 타임 체크)	동적 타입 (런타임에 자유롭게 변경)

```
// JS: Array 하나로 저장 + 동적 크기 + 처리 모두 가능
const nums = [3, 1, 4, 1, 5];
nums.push(9);
nums[0];
const result = nums.filter(n => n > 2).map(n => n * 10); // 처리 (Stream)

// 동적 크기 (ArrayList)
// 인덱스 접근 (int[])
```

```
// Java: 역할별로 분리
int[] arr = {3, 1, 4, 1, 5}; // 고정 크기 배열
List<Integer> list = new ArrayList<>(Arrays.asList(3, 1, 4, 1, 5)); // 동적 배열
List<Integer> result = list.stream()
    .filter(n -> n > 2)
    .map(n -> n * 10)
    .collect(Collectors.toList()); // 처리 파이프라인
```

2-2. 설계 차이에서 오는 구체적 차이

Java는 collect() 필요 vs JS는 바로 배열 반환

Java는 Stream이 별도 객체이므로 처리 후 `collect()`로 다시 컬렉션으로 변환해야 한다. JS는 배열 메서드가 바로 배열을 반환한다.

```
// JS: filter/map 결과가 바로 배열
const filtered = [1, 2, 3, 4].filter(n => n % 2 === 0); // [2, 4]
```

```
// Java: collect()로 수집해야 List가 됨
List<Integer> filtered = Arrays.asList(1, 2, 3, 4).stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList()); // [2, 4]
```

Java는 체이닝이 기본 vs JS는 끊어쓰기도 가능

Java는 Stream을 열면 체이닝으로 한 번에 이어써야 자연스럽다. 중간에 끊으려면 매번 `collect()`로 닫고 다시 `stream()`을 열어야 해서 비효율적이다. JS는 각 고차함수가 바로 배열을 반환하므로, 체이닝으로 이어쓰든 변수에 담아 끊어쓰든 자유롭다.

```
// JS: 끊어쓰기 가능 - 각 단계가 독립적인 배열
const scores = [45, 78, 92, 33, 88, 61, 55];
const filtered = scores.filter(s => s >= 60); // [78, 92, 88, 61]
const added = filtered.map(s => s + 10); // [88, 102, 98, 71]
const result = added.reduce((sum, s) => sum + s, 0); // 359
```

```
// Java: 끊어쓰면 매번 stream을 다시 열어야 함
List<Integer> filtered = scores.stream()
    .filter(s -> s >= 60)
    .collect(Collectors.toList()); // stream 종료
```

```
int result = filtered.stream() // 다시 stream 열어야 함
    .map(s -> s + 10)
    .reduce(0, Integer::sum);
```

Java Stream은 일회용 vs JS 배열은 재사용 가능

Java는 성능을 위해 Stream을 별도 객체로 분리한 결과, 한 번 소비하면 재사용할 수 없다.

```
// JS: 같은 배열로 여러 번 처리 가능
const arr = [1, 2, 3];
const sum = arr.reduce((a, b) => a + b, 0); // 6
const doubled = arr.map(n => n * 2); // [2, 4, 6] - 같은 arr 재사용
```

```
// Java: Stream은 한 번 쓰면 끝
// Stream<Integer> s = Arrays.asList(1, 2, 3).stream();
// s.reduce(0, Integer::sum); // 6
// s.map(n -> n * 2); // IllegalStateException - 재사용 불가!
```

Java는 타입 선언 필수 vs JS는 동적 타입

```
// JS: 타입 없이 자유롭게
const map = {};
map["key"] = 42;
map["key"] = "hello"; // 같은 키에 다른 타입 가능
```

```
// Java: 타입 명시 필수
Map<String, Integer> map = new HashMap<>();
map.put("key", 42);
// map.put("key", "hello"); // 컴파일 에러! Integer만 가능
```

2-3. 코딩테스트에서 자주 실수하는 부분

JS sort()는 기본이 문자열 정렬 (가장 흔한 실수)

Java는 숫자를 자동으로 숫자순 정렬하지만, JS는 콜백 없이 sort()를 호출하면 모든 요소를 문자열로 변환 후 사전순 정렬한다.

```
// JS: 콜백 없으면 문자열 정렬 - 반드시 콜백 필요!
[10, 9, 1, 2].sort(); // [1, 10, 2, 9] ← 틀림!
[10, 9, 1, 2].sort((a, b) => a - b); // [1, 2, 9, 10] ← 올바른
```

```
// Java: Collections.sort()는 Comparable 기반 (숫자는 자동 숫자순)
List<Integer> nums = new ArrayList<>(Arrays.asList(10, 9, 1, 2));
Collections.sort(nums); // [1, 2, 9, 10] - 문제 없음
```

2-4. 설계 차이가 코딩테스트 패턴에 미치는 영향

위의 설계 철학 차이가 코딩테스트 패턴에도 영향을 준다. 같은 유형의 문제라도 각 언어가 제공하는 도구가 달라서 자주 쓰이는 패턴이 달라진다.

같은 패턴 번호인데 내용이 다른 경우

자료 구조	패 턴	Java	JavaScript	이유
Array	패 턴 3	존재 여부 + 위치 확인 — contains / indexOf	찾아서 제거 — indexOf + splice	Java는 remove(Object o)가 있어서 "찾아서 제거"를 따로 조합할 필요 없음. JS는 값 기반 제거 메서드가 없어서 indexOf + splice 조합이 필수
Array	패 턴 4	모든 쌍 비교 — 이 중 반복문	2차원 배열 순회 — 이중 인덱스	같은 이중 반복이지만 각 언어에서 더 자주 출제되는 유형이 다름

한쪽에만 있는 패턴

자료구 조	Java	JavaScript	이유
Stream / 고차함 수	패턴 1: 중복 제거 + 정렬을 distinct() + sorted()로 스트림 안에서 처리	패턴 1: 고차함수 없이 Set + sort로 해결	JS는 Stream이 없어 Set 자료구조 로 대체
Stream / 고차함 수	findFirst(), anyMatch(), allMatch() 있지만 코테에서 빈도 낮 음	패턴 5~6: find, some, every 자주 사용	JS는 배열에 내장되어 접근이 쉬워 활용 빈도가 높음
Stream / 고차함 수	체이닝은 기본 동작이라 별도 패턴 없음	패턴 7: 메서드 체이닝	Java는 stream/collect 구조가 항 상 체이닝이라 별도로 다루지 않음

결과적으로

Java는 도구를 조합하는 패턴이 많고 (stream + collect, Comparator + thenComparing), JavaScript는 단일 메서드로 해결하는 패턴이 많다 (find, some, splice).

3. 같은 문제, 두 언어 — 코드 비교

비교 1: 빈도 세기

문제: 배열에서 각 요소의 등장 횟수를 구하라.

```
// Java: HashMap + getOrDefault
import java.util.*;

public class FrequencyCount {
    public static void main(String[] args) {
        String[] arr = {"a", "b", "a", "c", "b", "a"};

        Map<String, Integer> count = new HashMap<>();
        for (String s : arr) {
            count.put(s, count.getOrDefault(s, 0) + 1);
        }

        System.out.println(count); // {a=3, b=2, c=1}
    }
}
```

```
// JS: Object + || 0 패턴
const arr = ["a", "b", "a", "c", "b", "a"];

const count = {};
for (const s of arr) {
    count[s] = (count[s] || 0) + 1;
}

console.log(count); // { a: 3, b: 2, c: 1 }
```

비교 2: 커스텀 정렬

문제: 문자열 배열을 길이 오름차순으로, 같은 길이면 사전순으로 정렬하라.

```
// Java: Comparator + thenComparing
import java.util.*;

public class CustomSort {
    public static void main(String[] args) {
        List<String> words = new ArrayList<>(
            Arrays.asList("banana", "fig", "apple", "kiwi", "date")
        );

        words.sort(Comparator.comparingInt(String::length)
            .thenComparing(Comparator.naturalOrder()));

        System.out.println(words); // [fig, date, kiwi, apple, banana]
    }
}
```

```

    }
}

```

```

// JS: sort 콜백에서 || 연산자로 다단계
const words = ["banana", "fig", "apple", "kiwi", "date"];

words.sort((a, b) =>
  a.length - b.length || a.localeCompare(b)
);

console.log(words); // ["fig", "date", "kiwi", "apple", "banana"]

```

비교 3: 필터 + 변환 파이프라인

문제: 숫자 배열에서 짝수만 골라 제공한 리스트를 구하라.

```

// Java: Stream filter → map → collect
import java.util.*;
import java.util.stream.*;

public class FilterMap {
    public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5, 6);

        List<Integer> result = nums.stream()
            .filter(n -> n % 2 == 0) // [2, 4, 6]
            .map(n -> n * n)          // [4, 16, 36]
            .collect(Collectors.toList());

        System.out.println(result); // [4, 16, 36]
    }
}

```

```

// JS: filter → map 체이닝 (collect 불필요)
const nums = [1, 2, 3, 4, 5, 6];

const result = nums
  .filter(n => n % 2 === 0) // [2, 4, 6]
  .map(n => n * n);         // [4, 16, 36]

console.log(result); // [4, 16, 36]

```

4. 실무 관점

Java의 강점

- **타입 안정성**: 컴파일 타임에 오류를 잡을 수 있어 대규모 서비스에서 유리
- **엔터프라이즈 생태계**: Spring 기반 백엔드에서 압도적 점유율
- **멀티스레딩**: 병렬 처리와 동시성 제어에 강력한 도구 제공

JavaScript의 강점

- **프로토타이핑 속도**: 동적 타입 + 유연한 문법으로 빠르게 구현
- **풀스택 가능**: Node.js로 백엔드, React/Vue로 프론트엔드 — 한 언어로 전체 스택
- **비동기 처리**: 이벤트 루프 기반으로 I/O 집약적 작업에 효율적

두 언어를 모두 아는 것의 이점

- 백엔드(Java) ↔ 프론트엔드(JS) 간 소통 능력
- 한 언어의 개념을 다른 언어로 매핑할 수 있는 사고력
- 코딩테스트에서 문제에 따라 유리한 언어 선택 가능

5. 코딩테스트 전체 패턴 총정리

Array / ArrayList

패턴	Java	JavaScript
패턴 1: 스택처럼 사용	<code>add / get(size()-1)</code>	<code>push / pop</code>
패턴 2: 부분 배열 추출	<code>subList</code> (뷰 반환)	<code>slice</code> (복사본 반환)
패턴 3 (Java): 존재 여부 + 위치 확인	<code>contains / indexOf</code>	—
패턴 3 (JS): 찾아서 제거	—	<code>indexOf + splice</code>
패턴 4 (Java): 모든 쌍 비교	이중 반복문	—
패턴 4 (JS): 2차원 배열 순회	—	이중 인덱스

HashMap / Object

패턴	Java	JavaScript
패턴 1: 빈도 세기	<code>getOrDefault</code> vs <code>merge</code>	<code>obj[key] \ \ 0</code> vs <code>Map</code>
패턴 1 심화: 빈도 + 조합 계산	<code>entrySet</code> 순회 + <code>nC2</code>	<code>Object.entries</code> 순회 + 조합
패턴 2: 중복 제거 + 고유 개수	<code>HashSet</code> (심화: <code>HashMap + keySet</code>)	<code>Set</code> (심화: <code>Object + Object.keys</code>)
패턴 3: 부분 문자열 검사	<code>substring + HashSet.contains</code> <code>O(1)</code>	<code>substring + Set.has</code> <code>O(1)</code>

Comparator / sort + 콜백

패턴	Java	JavaScript
패턴 1: 기본 오름차순/내림차순	<code>Collections.sort / Comparator.reverseOrder()</code>	<code>sort((a, b) => a - b)</code>
패턴 2: 문자열 이어붙이기 커스텀 정렬	<code>(a, b) -> (b+a).compareTo(a+b)</code>	<code>(a, b) => (b+a) - (a+b)</code>
패턴 3: 내림차순 + 인덱스 조건 탐색	<code>reversed()</code>	콜백 부호 반전
패턴 4: 다단계 정렬	<code>thenComparing</code>	콜백 내 <code>\ \ </code> 다중 조건

Stream / 고차함수

패턴	Java	JavaScript
패턴 1: 중복 제거 + 정렬	<code>distinct + sorted + collect</code>	<code>[...new Set(arr)].sort()</code> (고차함수 없이)
패턴 2: 조건 필터링	<code>filter + collect</code>	<code>filter</code>
패턴 3: 변환	<code>map + collect</code>	<code>map</code>
패턴 4: 누적 계산	<code>reduce</code>	<code>reduce</code>
패턴 5: 검색	<code>findFirst</code> (빈도 낮음)	<code>find / findIndex</code>
패턴 6: 조건 확인	<code>anyMatch / allMatch</code> (빈도 낮음)	<code>some / every</code>
패턴 7: 체이닝	기본 동작 (stream→collect)	<code>filter().map().reduce()</code>